YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

Project – High Level Design
On
Personalized recommendationengine for smart city

Course: B.Sc. Artificial Intelligence, Machine Learning & Data Science

Student Name: Ahamed Mubaris Sayyan
Register No: 23BSAMD004
16/04/2026

Industry Mentor: Sumit Shukla
Guided By: Fathimath Raheema

TABLE OF CONTENTS

1.Introduction 3
1.1 Scope of the Document 3
1.2 Intended Audience 3
1.3 System Overview 3

System Design 4
2.1 Application Design 4
2.2 Process Flow 6
2.3 Information Flow 7
2.4 Components Design 7
2.5 Key Design Considerations 8
2.6 API Catalogue 8
3.Data Design 9
3.1 Data Model 9
3.2 Data Access Mechanism 9
3.3 Data Retention Policies 9
3.4 Data Migration 10
4.Interfaces 11
5.State and Session Management 12
6.Caching 13
7..Non-Functional Requirements 14
8.References 15

1. INTRODUCTION

A smart city uses technology and data to improve the daily life of people. One important problem in cities is that users often cannot easily find the best places to visit or food to eat. A personalized recommendation engine solves this problem by suggesting places based on user interests, location, and context such as weather.This system allows users to register, log in, and set their preferences such as food choices and places they like. Based on this data, the system provides recommendations using techniques like collaborative filtering and cosine similarity, which are

commonly discussed in research papers on recommender systems.The system also considers real-time data such as location and weather to improve the accuracy of suggestions. For example, on a rainy day, it may recommend indoor restaurants instead of outdoor places. Users can also give ratings and feedback, which helps improve future recommendations.

1.1 SCOPE OF THE DOCUMENT

This document describes the design and implementation of a smart city recommendation system. It includes details about system architecture, data flow, components, and technologies used such as Flask, HTML, CSS, JavaScript, and machine learning models

1.2 INTENDED AUDIENCE

This document is useful for:

- Students developing similar projects
- Developers working on smart city applications
- Researchers interested in recommendation system

1.3 SYSTEM OVERVIEW

The system overview gives a general idea of how the Personalized Recommendation Engine works. The system allows users to register, log in, and set their preferences based on their interests such as food and places. After setting preferences, the user can access a dashboard where recommendations are displayed.The system collects real-time data such as user

location and weather conditions. Based on this data and user preferences, the recommendation engine suggests nearby places or services. The system uses both context-based and collaborative filtering methods to provide accurate recommendations. Machine learning techniques like cosine similarity are used to compare user interests and find similar options.Users can also give ratings and feedback on the recommendations they receive. This feedback helps the system improve its future suggestions. The frontend of the system is designed using HTML, CSS, and JavaScript, while the backend is built using Python Flask.

2. SYSTEM DESIGN

2.1 APPLICATION DESIGN

This diagram shows the **complete architecture** of the Personalized Recommendation Engine for a Smart City. It explains how the user, frontend, backend, machine learning model, and database work together.

The process starts with the **User (Client)**, who uses a browser or mobile app to interact with the system. The user can perform actions like **registering, logging in, setting preferences (food, places), and giving ratings or feedback**.

These actions are handled by the **Frontend UI**, which is built using HTML, CSS, and JavaScript. The frontend provides different pages such as login, dashboard, and preference settings. It sends user requests to the backend using HTTP APIs.

The **Backend Server (Python Flask)** is the main control unit of the system. It handles authentication (login/register), processes user requests, and manages the recommendation engine. It also connects all components together.

The backend communicates with the **Machine Learning Model**, which is responsible for generating recommendations. It uses techniques like **collaborative filtering** and **cosine similarity** to compare user interests and find similar patterns. Based on this, it suggests relevant places or food items.

The system also interacts with the **Database**, which stores important data such as user details, preferences, and ratings. The backend fetches and updates this data when needed.

2.2 PROCESS FLOW

The process starts with the **Start** point. First, the user needs to **register** by creating an account.

After registration, the user proceeds to **login** using their credentials. Once the user enters login details, the system checks whether the user is valid or not.

If the user enters incorrect details, the system shows an **invalid input** message and sends the user back to the login step. This loop continues until the user provides correct login information. If the user is valid, the process moves forward.

Next, the user sets their **preferences**, such as interests in food, places, or activities. After that, the system **fetches context data**, which includes the user's current location and weather conditions. This information helps make recommendations more accurate and relevant.

Then, the **recommendation engine processes** the data using methods like context-based filtering and collaborative filtering. It uses machine learning techniques such as cosine similarity to find the best matches for the user.

After processing, the system **generates recommendations** and displays them on the dashboard. Finally, the user can **give ratings and feedback**, which helps improve future recommendations.

2.3 INFORMATION FLOW

Information flows between different components of the system. User input flows from the frontend to the backend through APIs. The backend processes this input and interacts with the database.The recommendation engine receives data such as user preferences, ratings, and contextual data. It computes similarity between users or items and generates recommendations.The results are sent back to the frontend and displayed to the user. Feedback from the user is again sent to the backend, creating a continuous learning loop.

2.4 COMPONENTS DESIGN

The system has the following components:

USER

- **Register/Login:** This module allows users to create an account and securely log in to the system.It stores basic details like username, email, and password in the database.

● **Set Preference**: This module collects and stores user interests such as favorite food and place and much moreIt helps the system understand user choices for better recommendations.Preferences are saved and updated whenever the user makes changes. **View Dashboard:** The dashboard is the main interface where users see recommendations and data.It displays location, weather, and nearby places ● dynamically.It provides a user-friendly view for interacting with the system. **Recommendation Engine**:This module generates personalized suggestions using machine learning algorithms. It uses techniques like collaborative filtering and ● cosine similarity. It considers user preferences, ratings, and context (location, weather). **Feedback**: This module allows users to rate and review recommended items.The feedback is stored in the database for future improvements. ●

2.5 KEY DESIGN CONSIDERATIONS

● Personalization accuracy
● Fast response time
● Easy user interface
● Scalability for large users

2.6 API CATALOGUE

The system uses APIs such as:

- Recommendation API
- Weather API
- Location API

3. DATA DESIGN

Data design explains how data is stored, organized, and used in the Personalized Recommendation Engine for Smart City. It is an important part of the system because all recommendations depend on user data. The system collects and manages different types of data such as user details, preferences, location, and feedback.The data is stored in a structured database so that it can be easily accessed and updated. Proper data design helps the system work efficiently and provide accurate recommendations. For example, when a user logs in and sets preferences, the system saves this information and uses it later to suggest relevant places or food.

3.1 DATA MODEL

User table

SL.no 1 2 3 4	VARCHAR	Primary Key	VARCHAR	null Not null
Column name	**size** 11 200	User id	VARCHAR	User name
user_id	**Constraints**	user_email	200 200	User Email
user_name	**Description**	user_password	Not null Not	User Password
Data type INT				

3.2 DATA ACCESS MECHANISM

Data is accessed using Flask APIs and database queries. The backend retrieves and updates data efficiently.

3.3 DATA RETENTION POLICIES

User data is stored securely and retained for improving recommendations. Old or unused data can be removed periodically.

3.4 DATA MIGRATION

If the system is upgraded, data can be migrated using database tools without loss of information.

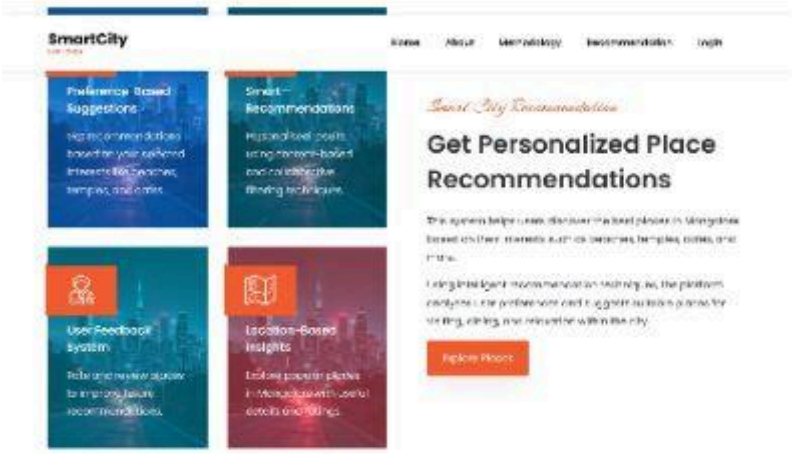
4. INTERFACES

Interfaces define how users and system components interact with the application. In this project, the main interface is the **user interface (UI)** developed using HTML, CSS, and JavaScript. It includes pages such as registration, login, preference setup, and dashboard. These pages are designed to be simple and user-friendly so that users can easily navigate the system.

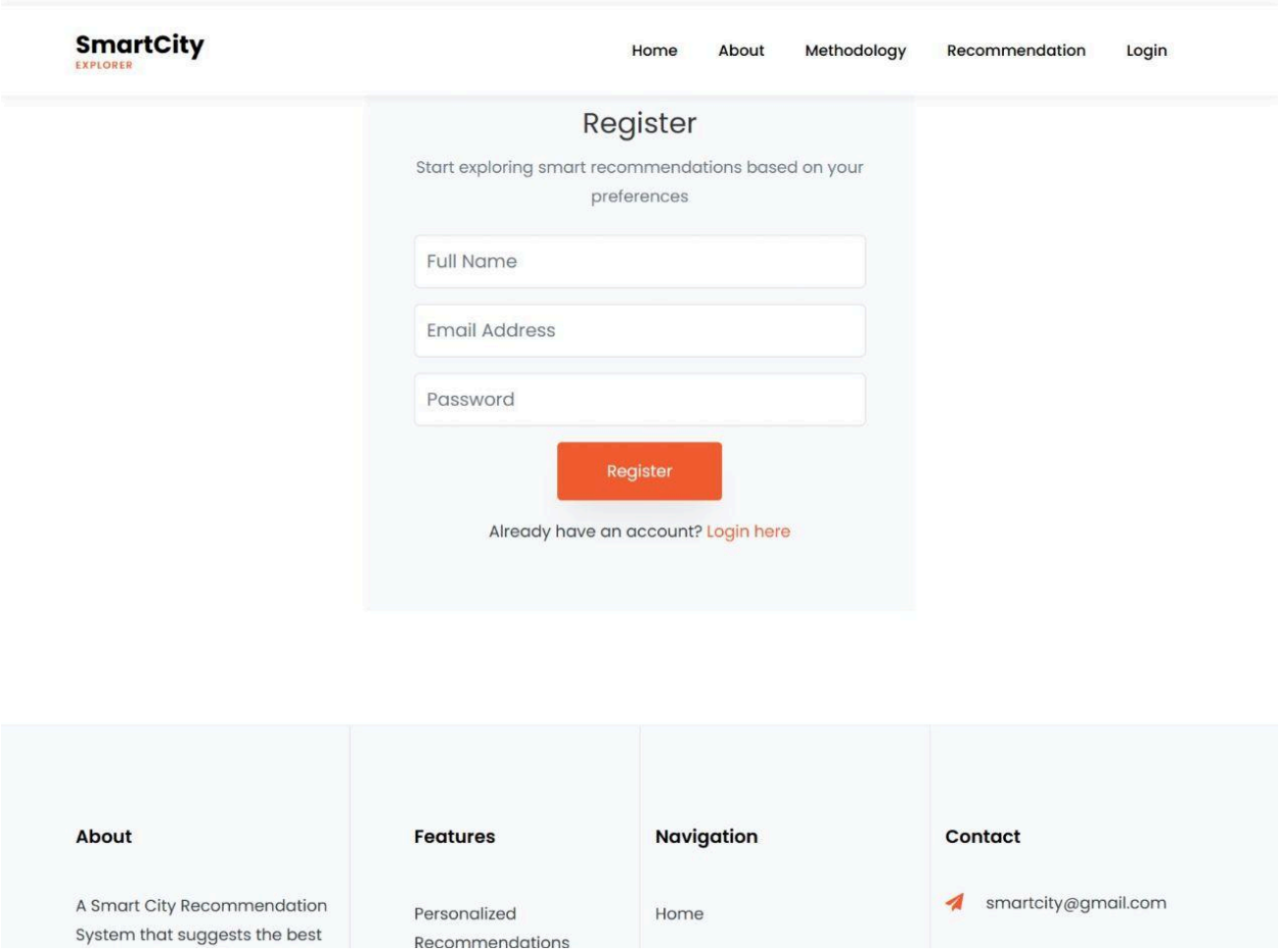
The system also uses **API interfaces** to connect the frontend and backend. The frontend sends requests (like login or fetching recommendations) to the Flask backend using HTTP methods such as GET and POST. The backend processes the request and sends a response back to the frontend.

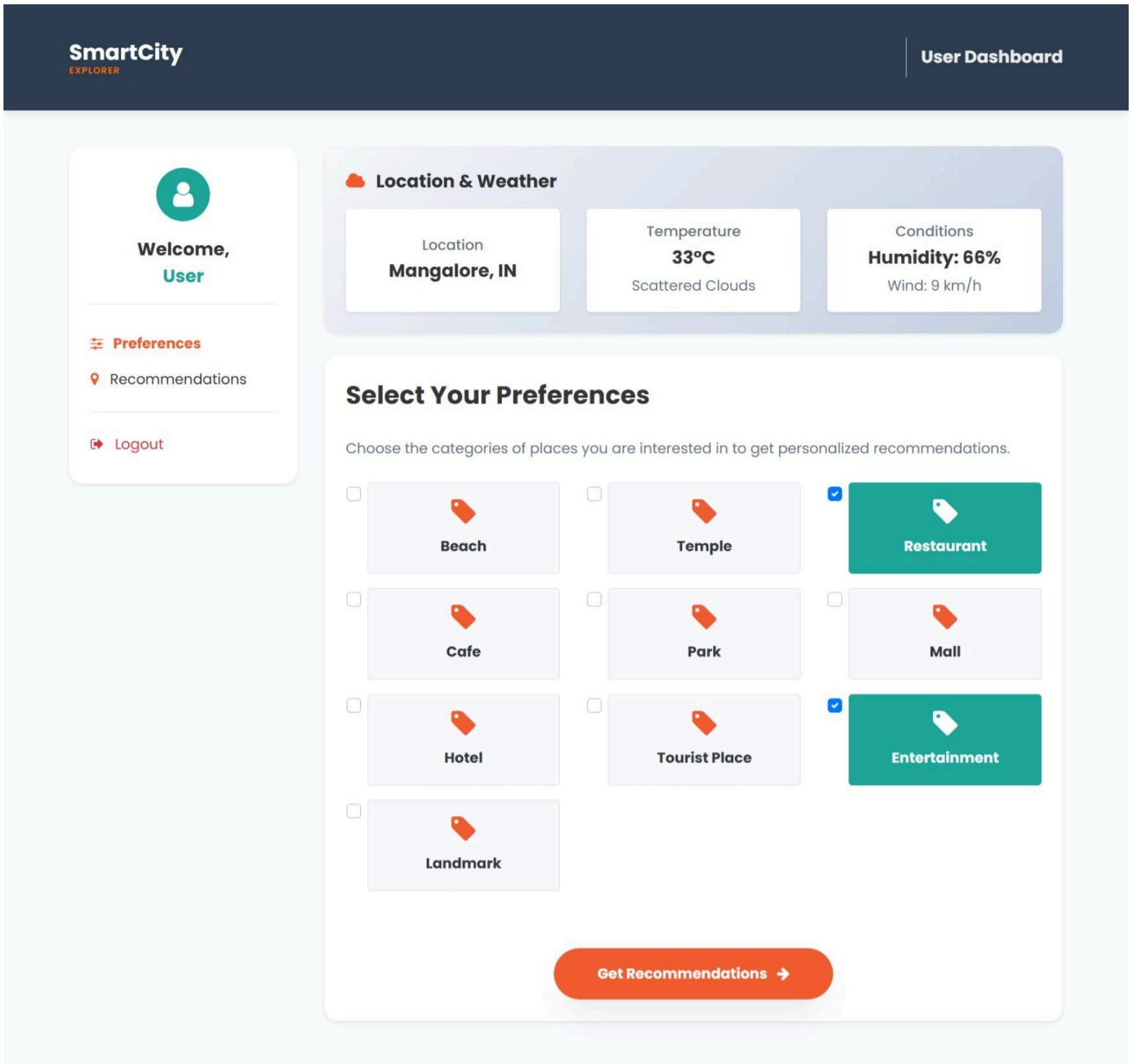
Additionally, the system integrates **external interfaces** such as weather and location services. These APIs provide real-time data which helps in generating context-aware recommendations. All interfaces work together to ensure smooth communication and better user experience.

Discover Personalized
Places in Mangalore


SmartCity
Preference Based Suggestions
Recommendations
User Feedback System
Location-Based Insights
Get Personalized Place
Recommendations

Create Account




SmartCity
EXPLORER
User Dashboard
Welcome,
User
Preferences
Recommendations
Logout
Location & Weather
Location
Mangalore, IN
Temperature
33°C
Scattered Clouds
Conditions
Humidity: 66%
Wind: 9 km/h
Select Your Preferences
Choose the categories of places you are interested in to get personalized recommendations.
Beach
Temple
Restaurant
Cafe
Park
Mall
Hotel
Tourist Place
Entertainment
Landmark
Get Recommendations

Location & Weather

Location
Mangalore, IN

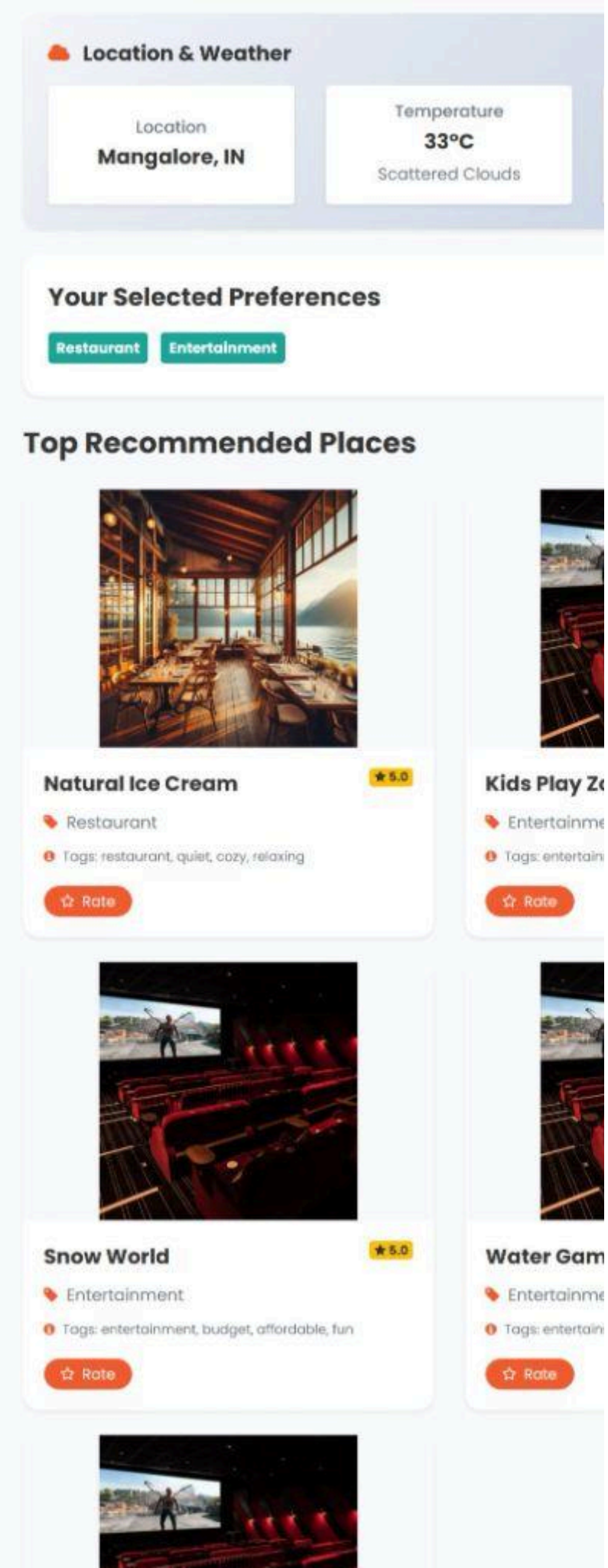
Temperature
33°C
Scattered Clouds

Select Your Preferences

Choose the categories of places you are interested in to get personalized

- [ ] Beach
- [ ] Temple
- [x]
- [ ] Cafe
- [ ] Park
- [ ]
- [ ] Hotel
- [ ] Tourist Place
- [x]
- [ ] Landmark

Get Recommendations →


Location & Weather
Location
Mangalore, IN
Temperature
33°C
Scattered Clouds
Your Selected Preferences
Restaurant
Entertainment
Top Recommended Places
Natural Ice Cream
5.0
Restaurant
Tags: restaurant, quiet, cozy, relaxing
Rate
Kids Play Z
Entertainme
Tags: entertain
Rate
Snow World
5.0
Entertainment
Tags: entertainment, budget, affordable, fun
Rate
Water Gam
Entertainme
Tags: entertain
Rate

5. STATE AND SESSION MANAGEMENT

State and session management are used to maintain user information during interaction with the system. When a user logs in, a session is created on the server using Flask. This session stores user-related data such as user ID and login status.

The session ensures that the user does not need to log in again while navigating between different pages. It helps in maintaining continuity and personalization across the application.

Secure session handling is important to protect user data. Techniques like session timeout and secure cookies are used to prevent unauthorized access. Proper session management improves bothsecurity and user experience.

6. CACHING

Caching is used to improve system performance by storing frequently accessed data temporarily. Instead of processing the same request multiple times, cached data can be quickly retrieved.

In this system, recommendations and frequently accessed user data can be cached. This reduces the load on the server and speeds up response time.

Caching can be implemented using in-memory storage or simple tools like Redis. It is especially useful when many users are using the system at the same time. Proper caching ensures faster performance and better scalability.

7. NON-FUNCTIONAL REQUIREMENTS

Non-functionalrequirements definehow the systemperformsrather than what it does. These include performance, security, usability, and scalability.

The system should be reliable and always available to users. It should handle multiple users without crashing. The interface should be simple and easy to use.

The system should also be scalable, meaning it can grow as more users join. Proper design ensures that the system continues to work efficiently even with increased data and usage.

- **Security Aspects:**Security is very important in this system because it handles user data. The system uses authentication to ensure only valid users can access their accounts.Passwords are stored securely using encryption techniques. APIs are protected to prevent unauthorized access. Session management is also secured to avoid attacks like session hijacking.
- **Performance Aspects:-** Performance refers to how fast and efficiently the system works. The system should provide quick responses when users request recommendations.Efficient algorithms like cosine similarity are used to reduce computation time. Database queries are optimized to fetch data quickly.

8.REFERENCES

[1]A Survey on RecommendationSystemsUsingMachineLearning – https://arxiv.org/abs/1902.06405

[2]Collaborative Filtering for Recommender Systems: A Survey –

https://dl.acm.org/doi/10.1145/3361682
[3]Content-Based Recommendation Systems: A Review – https://www.sciencedirect.com/science/article/pii/S1877050917315345
[4]Hybrid Recommendation Systems: A Systematic Review – https://ieeexplore.ieee.org/document/8126033
[5]Smart City Data Analytics and Recommendation Systems – https://www.mdpi.com/2071-1050/12/5/2004
[6]Machine Learning Techniques for Recommendation Engines – https://arxiv.org/abs/2006.10399
[7]Cosine Similarity Based Recommendation System – https://www.researchgate.net/publication/327894560